

Rojan Gabriel M. Calipusan

Bachelor of Science in Information Technology 2nd Year

21st Century IT Skills 2-C

Title: AI tool prompt engineering report

AI Overview

Name of the AI tool: ChatGPT Pro version

Purpose/domain: Code generation/ software development

Why you choose it?

The reason I chose it is its popularity and how easy the website is to use. Another reason I chose this tool is that it is relevant to my field as an IT student and is commonly used to assist with programming tasks, such as system development.

Original Prompt

Create a student enrollment system using Python. It should allow adding students, courses, and enrollment.

Output of the prompt:

Here's a clean Python student enrollment system using OOP. It lets you:

- add students
- add courses
- enroll a student in a course
- view students
- view courses
- view enrollments

You can copy this into a file like `enrollment_system.py` and run it.

class Student:

```
def __init__(self, student_id, name, age):  
    self.student_id = student_id  
    self.name = name  
    self.age = age
```

```
def __str__(self):  
    return f"ID: {self.student_id}, Name: {self.name}, Age: {self.age}"
```

class Course:

```
def __init__(self, course_code, course_name, units):  
    self.course_code = course_code  
    self.course_name = course_name  
    self.units = units
```

```
def __str__(self):  
    return f"Code: {self.course_code}, Course: {self.course_name}, Units:  
{self.units}"
```

```
class EnrollmentSystem:
```

```
    def __init__(self):  
        self.students = {}  
        self.courses = {}  
        self.enrollments = {}
```

```
    def add_student(self, student_id, name, age):  
        if student_id in self.students:  
            print("Student ID already exists.")  
            return  
        self.students[student_id] = Student(student_id, name, age)  
        print("Student added successfully.")
```

```
    def add_course(self, course_code, course_name, units):  
        if course_code in self.courses:  
            print("Course code already exists.")  
            return  
        self.courses[course_code] = Course(course_code, course_name, units)  
        print("Course added successfully.")
```

```
    def enroll_student(self, student_id, course_code):  
        if student_id not in self.students:  
            print("Student not found.")  
            return  
        if course_code not in self.courses:  
            print("Course not found.")  
            return
```

```
        if student_id not in self.enrollments:  
            self.enrollments[student_id] = []
```

```
        if course_code in self.enrollments[student_id]:  
            print("Student is already enrolled in this course.")  
            return
```

```

        self.enrollments[student_id].append(course_code)
        print("Student enrolled successfully.")

def view_students(self):
    if not self.students:
        print("No students available.")
        return
    print("\n--- Student List ---")
    for student in self.students.values():
        print(student)

def view_courses(self):
    if not self.courses:
        print("No courses available.")
        return
    print("\n--- Course List ---")
    for course in self.courses.values():
        print(course)

def view_enrollments(self):
    if not self.enrollments:
        print("No enrollments available.")
        return
    print("\n--- Enrollment List ---")
    for student_id, course_codes in self.enrollments.items():
        student = self.students[student_id]
        print(f"\nStudent: {student.name} ({student.student_id})")
        for code in course_codes:
            course = self.courses[code]
            print(f" - {course.course_name} ({course.course_code})")

def main():
    system = EnrollmentSystem()

    while True:
        print("\n===== Student Enrollment System =====")
        print("1. Add Student")
        print("2. Add Course")

```

```
print("3. Enroll Student")
print("4. View Students")
print("5. View Courses")
print("6. View Enrollments")
print("7. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
```

```
    student_id = input("Enter Student ID: ")
```

```
    name = input("Enter Student Name: ")
```

```
    age = input("Enter Student Age: ")
```

```
    if not age.isdigit():
```

```
        print("Age must be a number.")
```

```
        continue
```

```
    system.add_student(student_id, name, int(age))
```

```
elif choice == "2":
```

```
    course_code = input("Enter Course Code: ")
```

```
    course_name = input("Enter Course Name: ")
```

```
    units = input("Enter Units: ")
```

```
    if not units.isdigit():
```

```
        print("Units must be a number.")
```

```
        continue
```

```
    system.add_course(course_code, course_name, int(units))
```

```
elif choice == "3":
```

```
    student_id = input("Enter Student ID: ")
```

```
    course_code = input("Enter Course Code: ")
```

```
    system.enroll_student(student_id, course_code)
```

```
elif choice == "4":
```

```
    system.view_students()
```

```
elif choice == "5":
```

```
    system.view_courses()
```

```
elif choice == "6":
```

```
    system.view_enrollments()
```

```
elif choice == "7":
    print("Exiting system...")
    break

else:
    print("Invalid choice. Try again.")
```

```
if __name__ == "__main__":
    main()
```

How it works

Student stores student details.

Course stores course details.

EnrollmentSystem handles all operations:

- **add_student()**
- **add_course()**
- **enroll_student()**
- **view_students()**
- **view_courses()**
- **view_enrollments()**

Example flow

You can:

1. Add student: **2024-001, Juan Dela Cruz, 19**
2. Add course: **CS101, Intro to Programming, 3**
3. Enroll the student in **CS101**
4. View enrollments

One honest note

This version is solid for a basic school project, but it is still in-memory only. Once you close the program, the data is gone.

The smarter next step is to upgrade it with:

- MySQL or SQLite database
- update/delete functions
- search student/course
- GUI using Tkinter
- grade management

I can turn this into a MySQL-based version next, which is way better for a real project.

Running code:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

1. Add Student
2. Add Course
3. Enroll Student
4. View Students
5. View Courses
6. View Enrollments
7. Exit
Enter your choice: 1
Enter Student ID: 1234
Enter Student Name: Rojan Calipusan
Enter Student Age: 18
Student added successfully.

===== Student Enrollment System =====
1. Add Student
2. Add Course
3. Enroll Student
4. View Students
5. View Courses
6. View Enrollments
7. Exit
Enter your choice: 2
Enter Course Code: IT213
Enter Course Name: Computer Programming
Enter Units: 3
Course added successfully.

===== Student Enrollment System =====
1. Add Student
2. Add Course
3. Enroll Student
4. View Students
5. View Courses
6. View Enrollments
7. Exit
Enter your choice: 3
Enter Student ID: 1234
Enter Course Code: IT213
Student enrolled successfully.

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

7. Exit
Enter your choice: 3
Enter Student ID: 1234
Enter Course Code: IT213
Student enrolled successfully.

===== Student Enrollment System =====
1. Add Student
2. Add Course
3. Enroll Student
4. View Students
5. View Courses
6. View Enrollments
7. Exit
Enter your choice: 4

--- Student List ---
ID: 1234, Name: Rojan Calipusan, Age: 18

===== Student Enrollment System =====
1. Add Student
2. Add Course
3. Enroll Student
4. View Students
5. View Courses
6. View Enrollments
7. Exit
Enter your choice: █

```

AI-Assisted Prompt

Create a structured student enrollment system in Python using object-oriented programming.

Requirements:

- **Create classes for Student, Course, and Enrollment**
- **Each student should have ID, name, and course list**
- **Each course should have course ID, title, and units**
- **Implement functions to:**
 - **Add students**
 - **Add courses**
 - **Enroll a student in a course**
 - **Display all students and their enrolled courses**
- **Include input validation and error handling**
- **Use clear comments and clean code structure**

Optional:

- **Add a simple menu-driven interface for user interaction**

Output:

Absolutely. Here's a more structured OOP student enrollment system in Python that matches your requirements and includes a menu-driven interface, input validation, error handling, and clean comments.

class Course:

"""

Represents a course with a course ID, title, and number of units.

"""

def __init__(self, course_id, title, units):

self.course_id = course_id

self.title = title

self.units = units

def __str__(self):


```
return f'{self.course_id} - {self.title} ({self.units} units)'
```

```
class Student:
```

```
    """
```

```
    Represents a student with an ID, name, and a list of enrolled courses.
```

```
    """
```

```
    def __init__(self, student_id, name):
```

```
        self.student_id = student_id
```

```
        self.name = name
```

```
        self.course_list = []
```

```
    def add_course(self, course):
```

```
        """
```

```
        Adds a course to the student's enrolled course list
```

```
        if not already enrolled.
```

```
        """
```

```
        if course in self.course_list:
```

```
            raise ValueError(f"Student is already enrolled in {course.course_id}.")
```

```
        self.course_list.append(course)
```

```
    def __str__(self):
```

```
        return f'{self.student_id} - {self.name}'
```

class Enrollment:

"""

Manages students, courses, and enrollment operations.

"""

def __init__(self):

self.students = {}

self.courses = {}

def add_student(self, student_id, name):

"""

Adds a new student after validating inputs.

"""

if not student_id.strip():

raise ValueError("Student ID cannot be empty.")

if not name.strip():

raise ValueError("Student name cannot be empty.")

if student_id in self.students:

raise ValueError("Student ID already exists.")

self.students[student_id] = Student(student_id, name)

```

def add_course(self, course_id, title, units):
    """
    Adds a new course after validating inputs.
    """
    if not course_id.strip():
        raise ValueError("Course ID cannot be empty.")
    if not title.strip():
        raise ValueError("Course title cannot be empty.")
    if course_id in self.courses:
        raise ValueError("Course ID already exists.")

    try:
        units = int(units)
        if units <= 0:
            raise ValueError

    except ValueError:
        raise ValueError("Units must be a positive integer.")

    self.courses[course_id] = Course(course_id, title, units)

def enroll_student_in_course(self, student_id, course_id):
    """
    Enrolls a student in a course after checking existence.

```

```

"""

if student_id not in self.students:

    raise ValueError("Student ID not found.")

if course_id not in self.courses:

    raise ValueError("Course ID not found.")


student = self.students[student_id]
course = self.courses[course_id]
student.add_course(course)


def display_students(self):
    """
    Displays all students and their enrolled courses.
    """

    if not self.students:

        print("\nNo students found.")

        return


print("\n===== STUDENT LIST =====")

for student in self.students.values():

    print(f"\nStudent ID: {student.student_id}")

    print(f"Name      : {student.name}")

```

```
if not student.course_list:

    print("Courses : No enrolled courses")

else:

    print("Courses :")

    for course in student.course_list:

        print(f" - {course}")
```

```
def display_menu():

    """

    Displays the menu options.

    """

    print("\n===== STUDENT ENROLLMENT SYSTEM =====")

    print("1. Add Student")

    print("2. Add Course")

    print("3. Enroll Student in Course")

    print("4. Display All Students")

    print("5. Exit")
```

```
def main():

    """

    Main program loop with menu-driven interface.
```

"""

system = Enrollment()

while True:

display_menu()

choice = input("Enter your choice (1-5): ").strip()

try:

if choice == "1":

print("\n--- Add Student ---")

student_id = input("Enter student ID: ").strip()

name = input("Enter student name: ").strip()

system.add_student(student_id, name)

print("Student added successfully.")

elif choice == "2":

print("\n--- Add Course ---")

course_id = input("Enter course ID: ").strip()

title = input("Enter course title: ").strip()

units = input("Enter number of units: ").strip()

system.add_course(course_id, title, units)

```
print("Course added successfully.")
```

```
elif choice == "3":
```

```
print("\n--- Enroll Student in Course ---")
```

```
student_id = input("Enter student ID: ").strip()
```

```
course_id = input("Enter course ID: ").strip()
```

```
system.enroll_student_in_course(student_id, course_id)
```

```
print("Student enrolled successfully.")
```

```
elif choice == "4":
```

```
system.display_students()
```

```
elif choice == "5":
```

```
print("Exiting program...")
```

```
break
```

```
else:
```

```
print("Invalid choice. Please enter a number from 1 to 5.")
```

```
except ValueError as error:
```

```
print(f"Error: {error}")
```

```
except Exception as error:
```

```
print(f"Unexpected error: {error}")
```

```
if __name__ == "__main__":
```

```
    main()
```

What this program does

This version is cleaner and more aligned with what instructors usually want in an OOP activity.

It has:

- **Student** class
- **Course** class
- **Enrollment** class
- validation for empty inputs
- duplicate checking
- unit validation
- prevention of duplicate course enrollment
- menu-driven interface

OOP structure

Here's the logic:

Student

- stores **student_id**
- stores **name**
- stores **course_list**

Course

- stores **course_id**
- stores **title**
- stores **units**

Enrollment

- manages all students and courses
- handles adding and enrolling
- displays student records

Example usage

You can test it like this:

- Add student: **S001, Juan Dela Cruz**
- Add course: **CS101, Introduction to Programming, 3**
- Enroll **S001** in **CS101**
- Display all students

Why this is better than a rushed basic version

Because this one actually has structure:

- responsibilities are separated by class
- errors are handled properly
- duplicate data is prevented
- the code is easier to explain during demo or oral defense

If you want to level it up

The next best upgrades would be:

- add remove student
- add remove course
- add search student
- add total enrolled units
- save data using file handling or MySQL/SQLite

Output of the code:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Rojan\Downloads\IT_skills> python -u "c:\User

===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 1

--- Add Student ---
Enter student ID: 1234
Enter student name: Rojan Calipusan
Student added successfully.

===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 2

--- Add Course ---
Enter course ID: IT213
Enter course title: IT Skills
Enter number of units: 3
Course added successfully.

===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 3

--- Enroll Student in Course ---
Enter student ID: 1234
Enter course ID: IT213
Student enrolled successfully.
```

```
===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 3

--- Enroll Student in Course ---
Enter student ID: 1234
Enter course ID: IT213
Student enrolled successfully.

===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 4

===== STUDENT LIST =====

Student ID: 1234
Name      : Rojan Calipusan
Courses   :
    - IT213 - IT Skills (3 units)

===== STUDENT ENROLLMENT SYSTEM =====
1. Add Student
2. Add Course
3. Enroll Student in Course
4. Display All Students
5. Exit
Enter your choice (1-5): 5
Exiting program...
PS C:\Users\Rojan\Downloads\IT_skills>
```

Output Comparison and Analysis

The original prompt was able to produce a working student enrollment system. It included the basic features such as adding students, adding courses, and enrolling students, along with a simple menu-driven interface. This made the program usable and easy to test. However, the overall design was quite basic. Although it used Object-Oriented Programming (OOP), the structure of the classes was not very clear, and the responsibilities were somewhat mixed. In addition, input validation and proper error handling were limited, which made the system feel more like a simple demonstration rather than a fully developed program.

On the other hand, the output generated from the improved prompt showed a noticeable improvement in both structure and quality. Because the prompt clearly specified the requirements, such as defining class responsibilities, adding validation, and maintaining clean code, the AI was able to produce a more organized and professional solution. The roles of the Student, Course, and Enrollment classes were much clearer, which made the code easier to read and

understand. The use of proper validation, prevention of duplicate entries, and structured error handling using exceptions also made the system more reliable and realistic.

The improved prompt had a strong impact on the overall output. In terms of clarity, the instructions were more specific, which helped reduce confusion and ambiguity. In terms of depth, the program included more features and safeguards, making it more complete. In terms of relevance, the system was closer to what would be expected in a real-world application rather than just a basic classroom example.

Overall, this activity showed that the way a prompt is written greatly affects the quality of the output. A simple prompt leads to a basic result, while a detailed and structured prompt produces a more complete and professional system. This highlights the importance of prompt engineering, especially in technical fields like software development where clear and precise instructions are essential.